KubeCon | CloudNativeCon

Europe 2021

Virtual

Cert-Manager Beyond Ingress – Exploring the Variety of Use Cases

Matthew Bates, Jetstack

Presented by
Matthew Bates, CTO

Cert-Manager Beyond Ingress

Exploring the Variety of Use Cases

Jetstack / cert-manager

Open source machine identity management and automation software for Kubernetes and OpenShift cloud native platforms

7K+	280
GitHub stars	Contributors

Wide adoption and contribution from across the community ecosystem

Now part of the Cloud Native Computing Foundation (CNCF) 'Sandbox'

CA -agnostic and highly extensible, including ACME, Venafi, HashiCorp Vault, Google CAS, Cloudflare..

cert-manager.io

Jetstack / cert-manager

Open source machine identity management and automation software for Kubernetes and OpenShift cloud native platforms

7K+	280
GitHub stars	Contributors

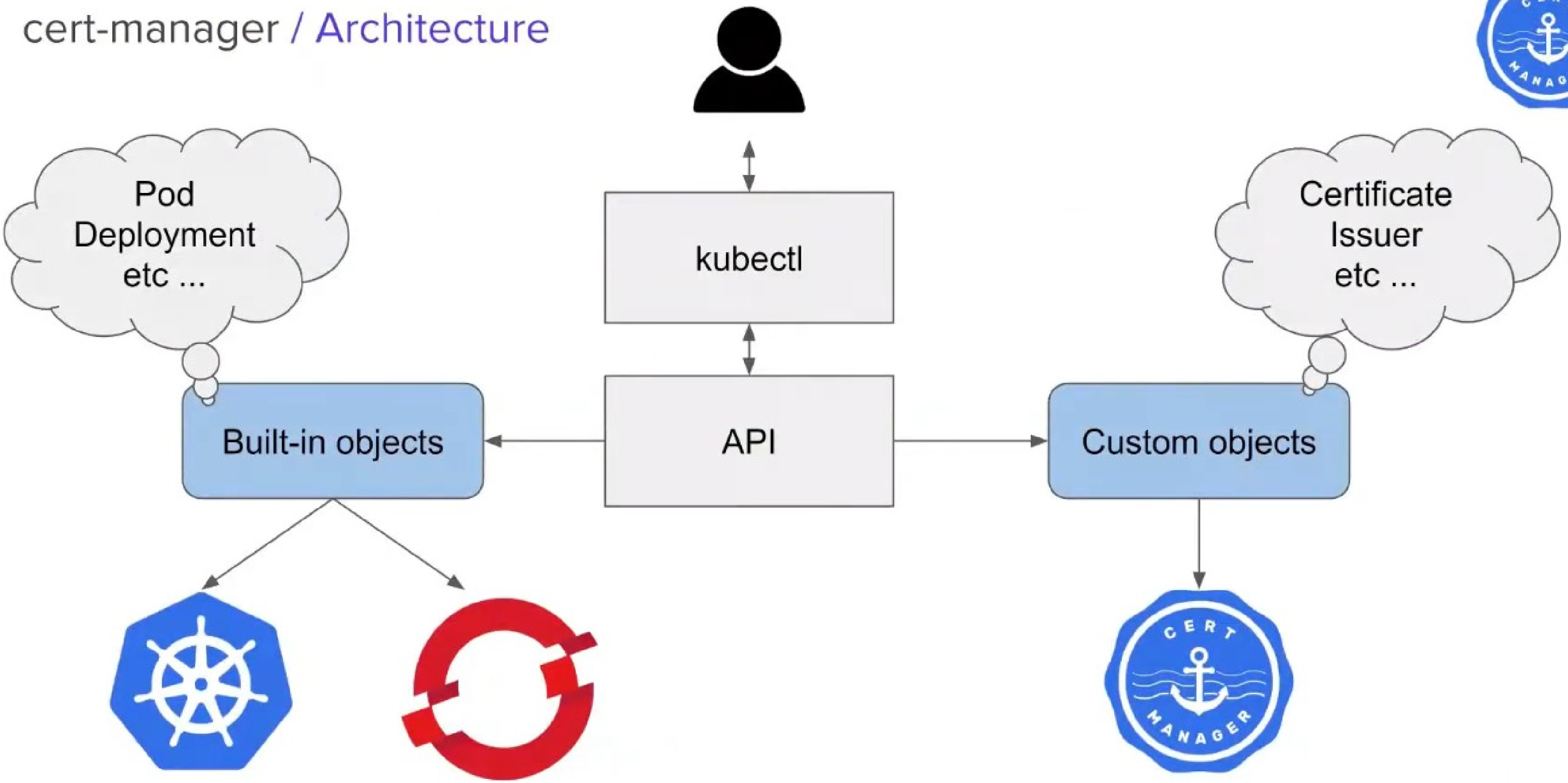
Wide adoption and contribution from across the community ecosystem

Now part of the Cloud Native Computing Foundation (CNCF) 'Sandbox'

CA -agnostic and highly extensible, including ACME, Venafi, HashiCorp Vault, Google CAS, Cloudflare..

cert-manager.io

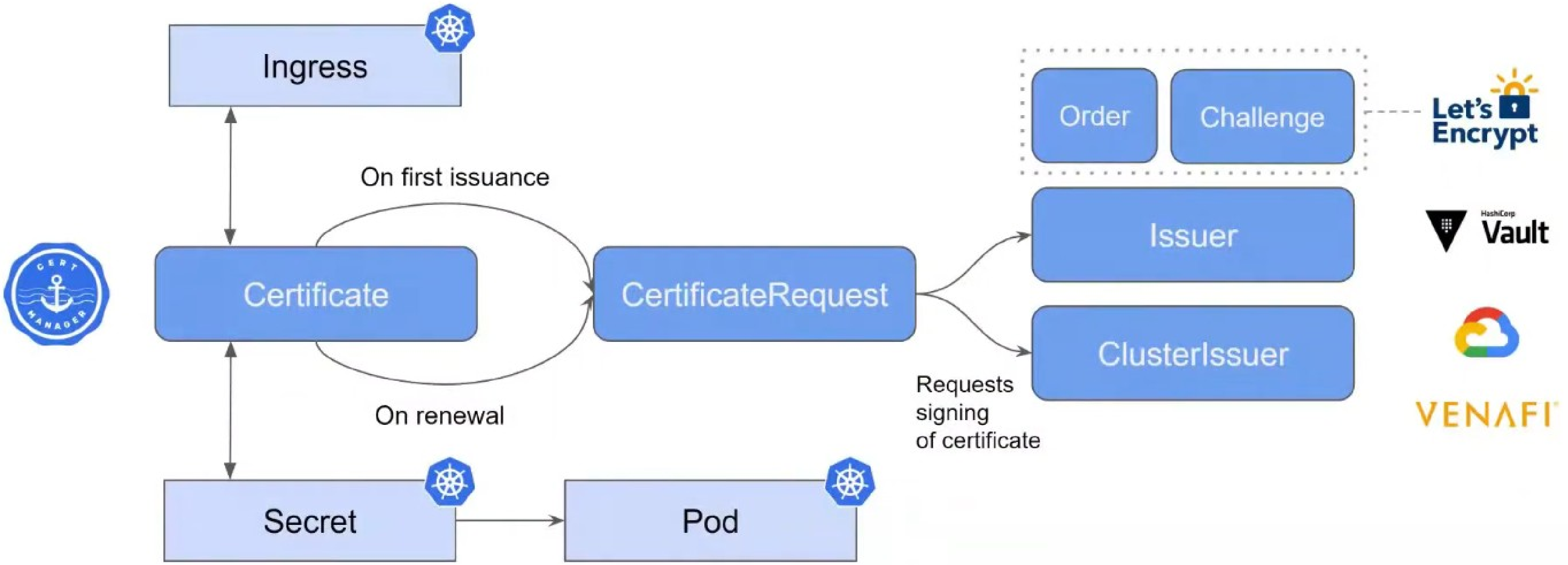
cert-manager / Architecture



cert-manager / Quick and easy certificates

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    kubernetes.io/ingress.class: "nginx"
    cert-manager.io/issuer: "letsencrypt-staging" # < how/where to obtain the cert
  name: myIngress
  namespace: myIngress
spec:
  rules:
  - host: myingress.com
    http:
      paths:
      - backend:
          serviceName: myservice
          servicePort: 80
        path: /
  tls: # < host in the TLS config indicates a certificate should be created
  - hosts:
    - myingress.com
    secretName: myingress-cert # < cert-manager will store the created certificate
```

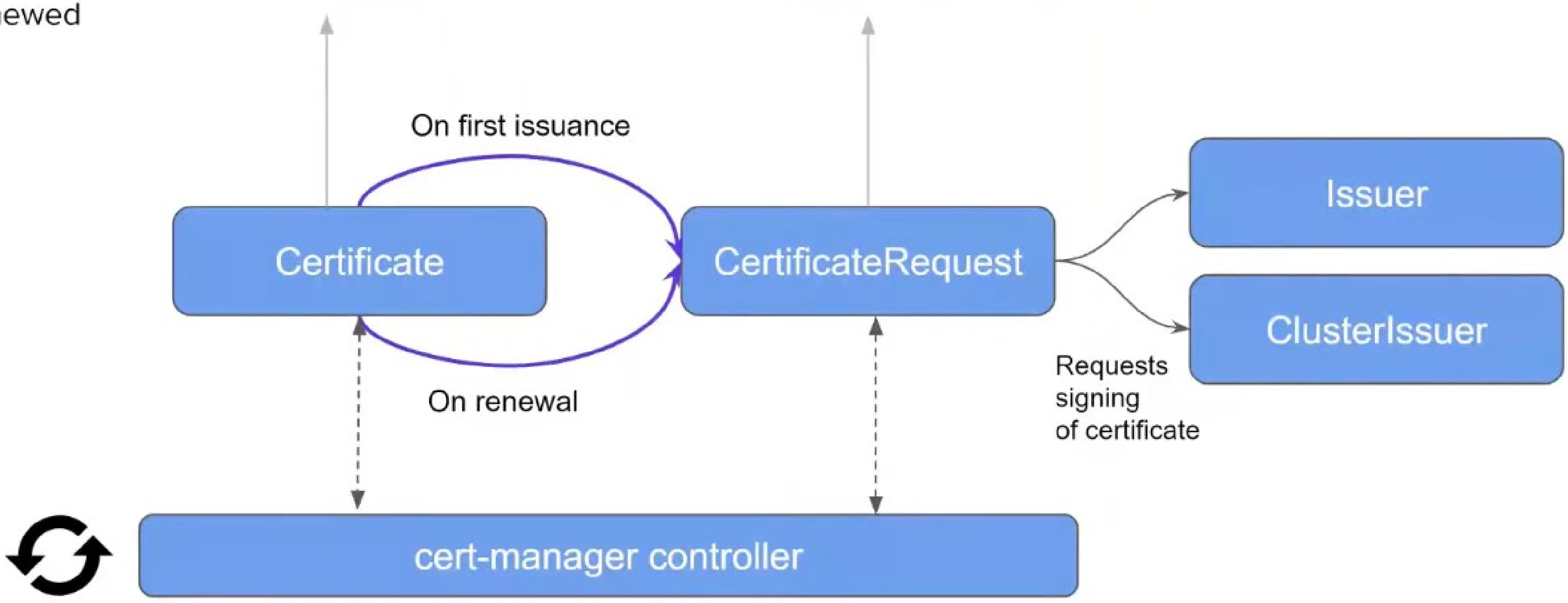
Ingress / Certificate Lifecycle



Ingress / Certificate Lifecycle

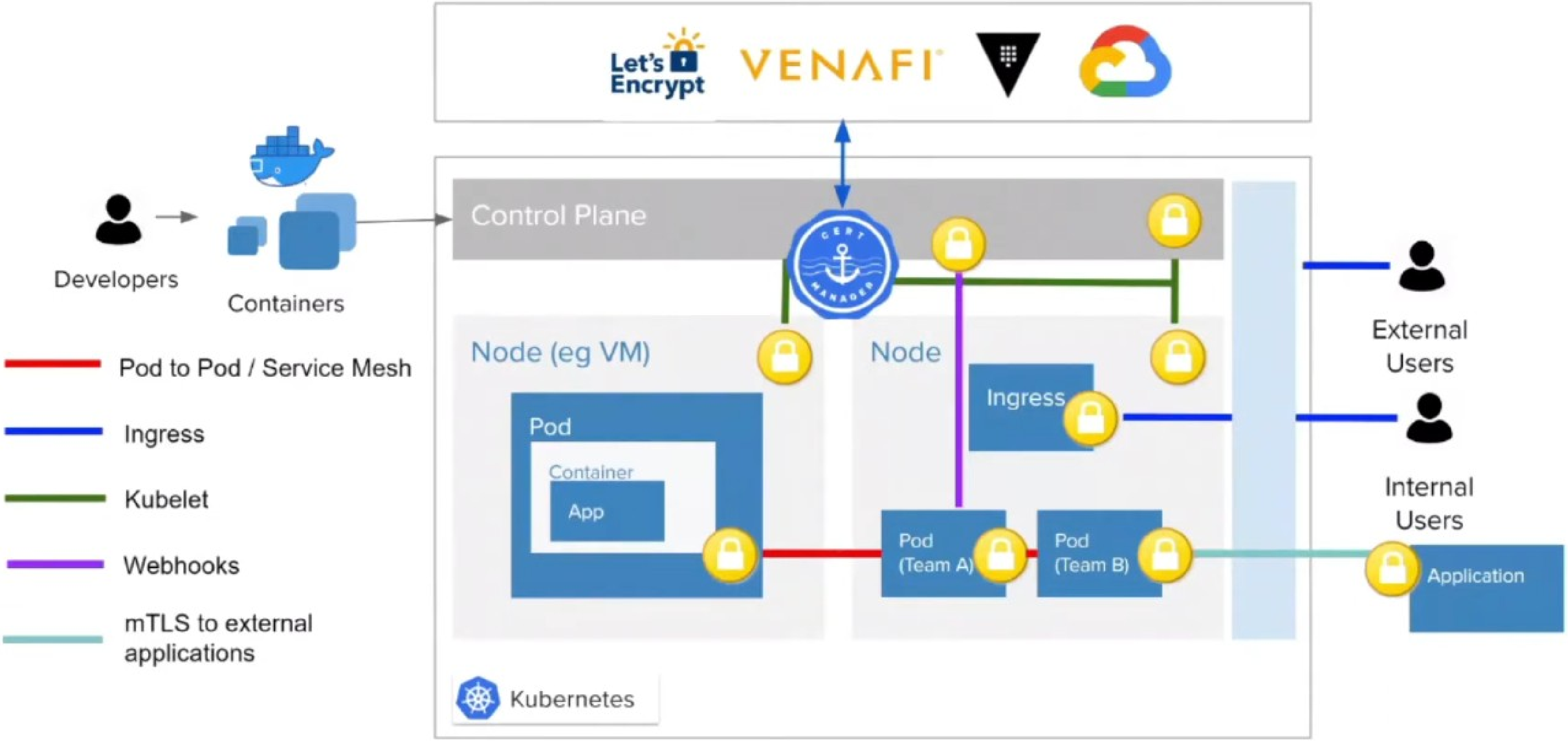
Certificate resource represents a human readable definition of a certificate that will be issued and automatically renewed

Point-in-time request for X.509 certificate from an Issuer. Typically consumed and managed by controllers or other systems



But there are lots of other certificates to manage in Kubernetes

cert-manager / Use cases

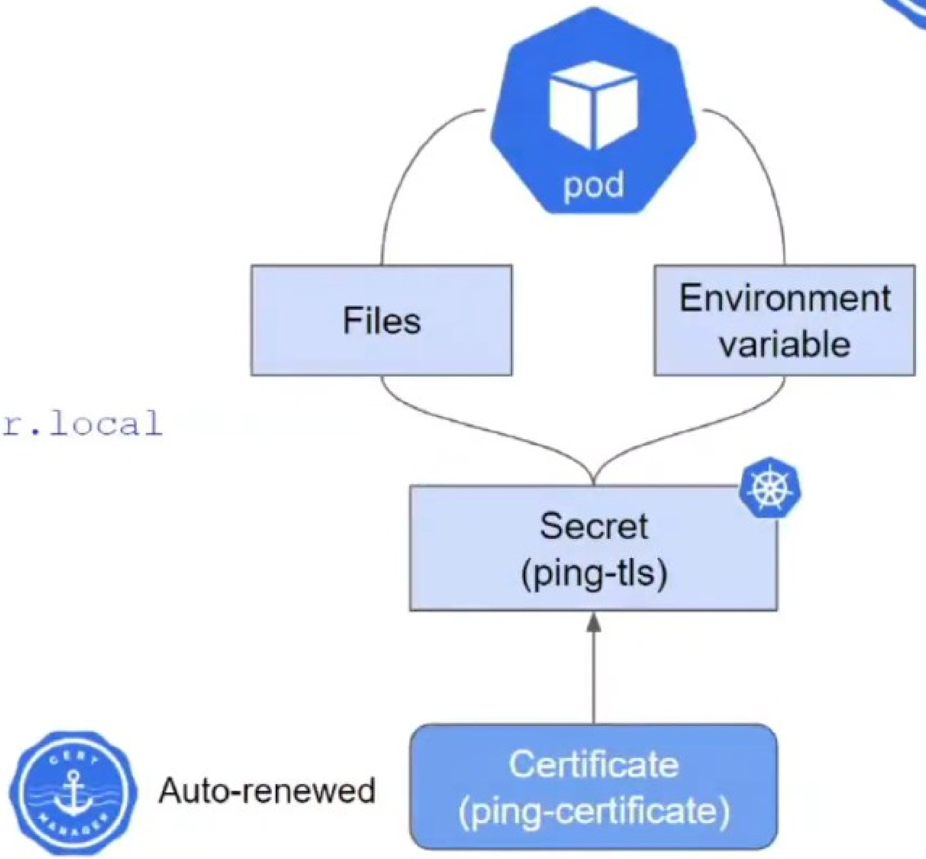


Using cert-manager for secure inter-pod communication ('east-west')

(without a service mesh 🤫)

cert-manager / Protecting applications (Workloads)

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: ping-certificate
spec:
  renewBefore: 30m
  secretName: ping-tls
  dnsNames:
    - ping-service.default.svc.cluster.local
  issuerRef:
    name: my-ca
    kind: Issuer
```



cert-manager / Protecting applications (Workloads)

```
...
spec:
      containers:
        - image: demo.cert-manager.io/pingpong:local
          name: pingpong
          command:
            - /usr/local/bin/pingpong
            - -endpoint=https://pong-service.default.svc.cluster.local:8443/ping
            - -ca-file=/etc/ssl/private/ca.crt
            - -cert-file=/etc/ssl/private/tls.crt
            - -key-file=/etc/ssl/private/tls.key
          volumeMounts:
            - mountPath: "/etc/ssl/private"
              name: ping-tls
              readOnly: true
          ports:
            - containerPort: 8443
              name: internal-https
            - containerPort: 9443
              name: external-https
      volumes:
        - name: ping-tls
          secret:
            secretName: ping-tls
```

cert-manager / Protecting applications (Workloads)

```
...
spec:
      containers:
        - image: demo.cert-manager.io/pingpong:local
          name: pingpong
          command:
            - /usr/local/bin/pingpong
            - -endpoint=https://pong-service.default.svc.cluster.local:8443/ping
            - -ca-file=/etc/ssl/private/ca.crt
            - -cert-file=/etc/ssl/private/tls.crt
            - -key-file=/etc/ssl/private/tls.key
          volumeMounts:
            - mountPath: "/etc/ssl/private"
              name: ping-tls
              readOnly: true
          ports:
            - containerPort: 8443
              name: internal-https
            - containerPort: 9443
              name: external-https
      volumes:
        - name: ping-tls
          secret:
            secretName: ping-tls
```

cert-manager / Private CA issuers

```yaml
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned
spec:
  selfSigned: {}

apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: my-ca
spec:
  ca:
    secretName: my-ca
```

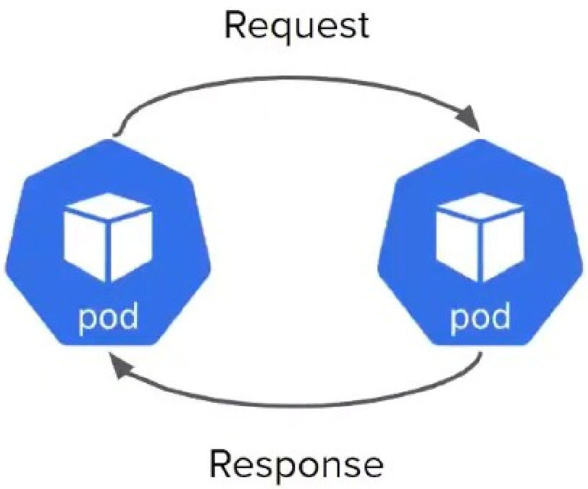
```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: my-ca
spec:
  isCA: true
  duration: 2160h # 90d
  secretName: my-ca
  commonName: my-ca
  subject:
    organizations:
    - cert-manager
  issuerRef:
    name: selfsigned
    kind: Issuer
    group: cert-manager.io
```

Built-in Issuers can be combined to create a self-signed CA

.. or plug in a core / external issuer

cert-manager / Inter-pod mutual TLS (mTLS)

- User calls Ping in the browser
- Ping contacts Pong and verifies itself with a client cert
- Pong replies and is secured using a server cert
- Ping replies to the browser with Pong's certificate info
- Ping and Pong only trust certificates signed by the same root CA



PingPong

https://github.com/jetstack/cert-manager-nginx-plus-lab

cert-manager / Inter-pod mutual TLS (mTLS)


```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: ping-certificate
spec:
  …
  secretName: ping-tls
  usages:
    - digital signature
    - key encipherment
    - client auth
    - server auth
  dnsNames:
    - ping-service.default.svc.cluster.local
  issuerRef:
    name: my-ca
    kind: Issuer
```

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: pong-certificate
spec:
  …
  secretName: pong-tls
  usages:
    - digital signature
    - key encipherment
    - client auth
    - server auth
  dnsNames:
    - pong-service.default.svc.cluster.local
  issuerRef:
    name: my-ca
    kind: Issuer
```

Configuring MySQL SSL/TLS authentication with cert-manager

Written by Maria Reynoso

https://www.jetstack.io/blog/securing-mysql-with-cert-manager/

Using cert-manager for secure intra-pod communication ('east-west')

(more seamlessly)

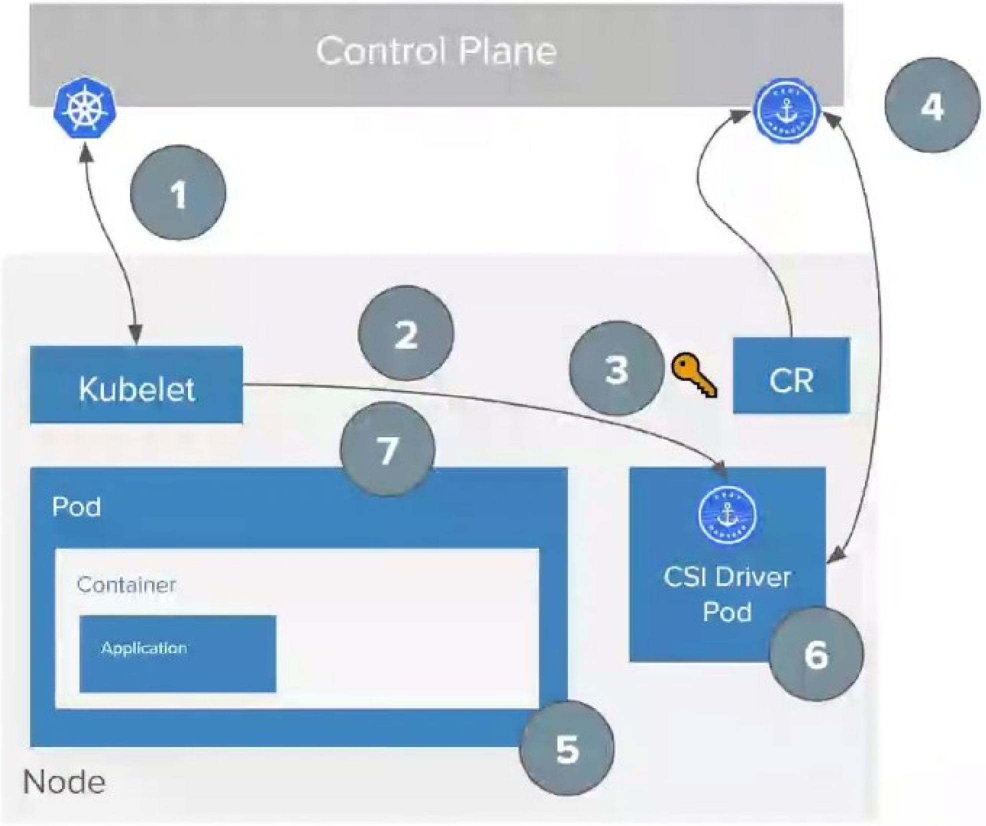
cert-manager / Protecting applications / CSI driver

jetstack / cert-manager-csi

- Ensure private keys never leave the node and are never sent over the network. All private keys are stored locally on the node.
- Unique key and certificate per application replica with a guarantee to be present on application run time.
- Reduce resource management overhead by defining certificate request spec in-line of the Kubernetes Pod template.
- Automatic renewal of certificates based on expiry of each individual certificate.
- Keys and certificates are destroyed during application termination.
- Scope for extending plugin behaviour with visibility on each replica's certificate request and termination.

https://github.com/jetstack/cert-manager-csi
https://cert-manager.io/docs/usage/csi/

cert-manager / Protecting applications / CSI driver flow



1. Pod scheduled to Node
2. Kubelet calls NodePublishVolume
3. Generate private key and CertificateRequest
4. cert-manager reconciles the CR and obtains a signed certificate
5. Signed certificate and private key will be written to that node's file system and mounted to the pod's file system.
6. Driver keeps track of certificates and auto-renews
7. NodeUnpublishVolume on termination

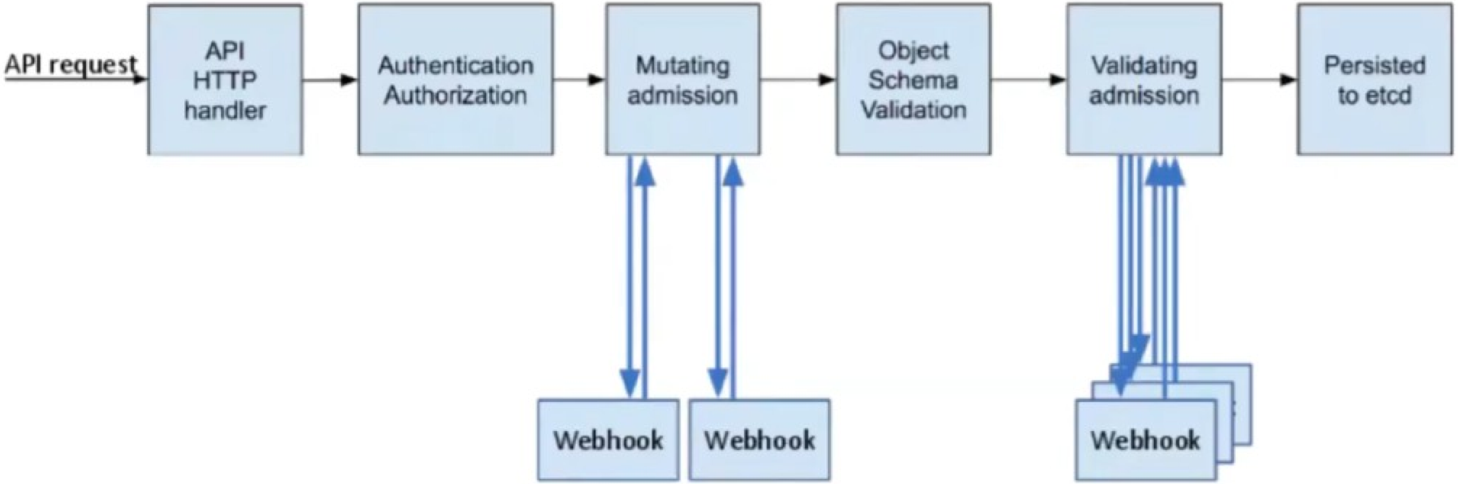
Using cert-manager to secure Kubernetes webhooks

Cert-manager / Securing Kubernetes webhooks

Powerful extension mechanism that can be used with built-in types as well as CRDs

Example uses:

- Applying default values
- Enforcing organisation policy
- Ensuring submitted resources are semantically valid



Credit: kubernetes.io

Cert-manager / Securing Kubernetes webhooks

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingWebhookConfiguration
metadata:
  name: webhook1
  annotations:
    cert-manager.io/inject-ca-from: example1/webhook1-certificate
    #cert-manager.io/inject-ca-from-secret: example2/example-ca
    #cert-manager.io/inject-apiserver-ca: "true"
 example1/webhook1-certificate
webhooks:
  ...
```

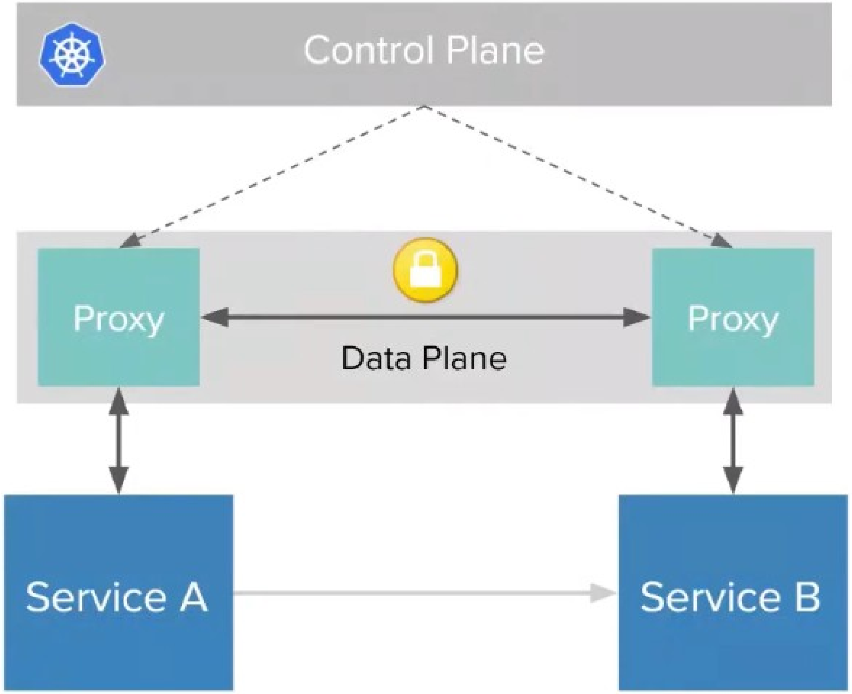
Cert-manager / Securing Kubernetes webhooks / Cluster API

```
$ clusterctl init --infrastructure docker
$ Fetching providers
$ Installing cert-manager Version="v1.1.0"
$ Waiting for cert-manager to be available...
...

$ kubectl get certificates -n capi-webhook-system
NAME                                        READY   SECRET                                               AGE
capi-kubeadm-bootstrap-serving-cert         True    capi-kubeadm-bootstrap-webhook-service-cert          19s
capi-kubeadm-control-plane-serving-cert     True    capi-kubeadm-control-plane-webhook-service-cert      17s
capi-serving-cert                           True    capi-webhook-service-cert                            20s
```

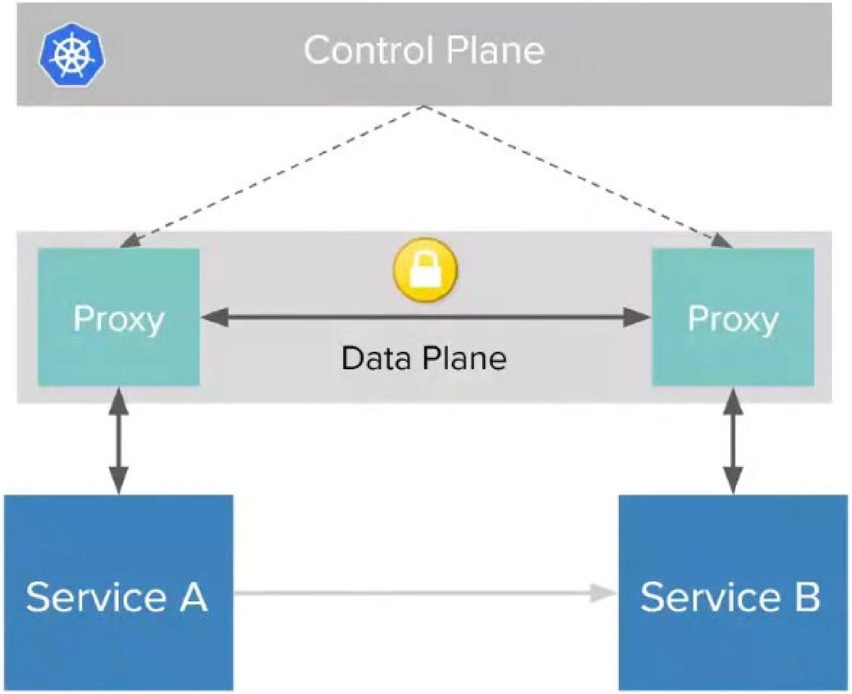
Using cert-manager to secure services in a mesh

cert-manager / Service mesh



- Service meshes provide the tools for observability, security, and reliability features to applications
- These capabilities are built-in the platform and the behaviour is programmable and dynamic.
- Developers need not implement at the application layer

cert-manager / Service mesh



- Service meshes provide the tools for observability, security, and reliability features to applications
- These capabilities are built-in the platform and the behaviour is programmable and dynamic.
- Developers need not implement at the application layer

cert-manager / Service mesh integrations

LINKERD

- A trust anchor, and an issuer certificate and private key, can be provided by cert-manager
- Linkerd manages workload certificates

- Plug in custom CA certificates with Citadel.
- istio-csr can replace Citadel as RA and integrate cert-manager
- K8s CSR API integration (*experimental*)

Open Service Mesh

- Pluggable certificate management including built- in component
- Integrates with cert-manager; OSM creates CertificateRequests

Using cert-manager to secure the Kubernetes control plane and nodes

- Lots of PKI involved in securing a Kubernetes cluster
  - Client certificates for the kubelet to authenticate to the API server
  - Server certificate for the API server endpoint
  - Client certificates for administrators of the cluster to authenticate to the API server
  - Client certificates for the API server to talk to the kubelets
  - Client certificate for the API server to talk to etcd
  - Client certificate/kubeconfig for the controller manager to talk to the API server
  - Client certificate/kubeconfig for the scheduler to talk to the API server.
  - Client and server certificates for the front-proxy
  - Note: front-proxy certificates are required only if you run kube-proxy to support an extension API server.
  - etcd also implements mutual TLS to authenticate clients and peers

cert-manager / Securing Kubernetes / Control plane

- **kubeadm** to the rescue - auto-generated and renewed control plane certs
- By default, uses a self-signed certificate authority (CA)

External CA mode

- kubeadm can also produce certificate signing requests (CSRs) for signing elsewhere

```
$ kubeadm certs generate-csr \
        --config kubeadm.conf \
        --kubeconfig-dir etc_kubernetes/pki \
        --cert-dir etc_kubernetes/pki
```

cert-manager / Securing kubeadm / Nodes

```yaml
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: node-certificate
spec:
  ...
  request: ....
  signerName: kubernetes.io/kube-apiserver-client-kubelet
  Usages:
  - key encipherment
  - digital signature
  - client auth
status:
  conditions:
  - message: Approved by approver controller
    ...
    type: Approved
```

Node certificates are signed with the Certificates API (certificates.k8s.io/v1)

Signers are built in to the kube-controller-manager.

https://github.com/cert-manager/signer-ca

https://github.com/cert-manager/signer-venafi

👀 Support for CertificateSigningRequest incoming in cert-manager

cert-manager / Summary

- Ingress (ingress-shim built into cert-manager)
- Many uses cases that stretch far and wide across the ecosystem
    - Use the Certificate / CertificateRequest resources directly
    - CSI driver for per-pod, auto-renewed certificates
    - Integrate with service meshes (eg Linkerd, Istio, OSM)
    - Kubernetes webhooks
    - Control plane and node certificates
- Support for the Kubernetes Certificates API
- Come and join us and get involved!
    - #cert-manager-dev #cert-manager on the K8s Slack

Thank you!

Slack: #cert-manager #cert-manager-dev

Web: cert-manager.io

Twitter: @JetstackHQ @mattbates25